Group 8: Automated Gravity-Fed Object Counter and Jam Detection System

TEAM DETAILS

S. NO.	TEAM MEMBER	REGISTRATION NUMBER
1	Anish Reddy	CB.SC.U4CSE24524
2	V. Rohit	CB.SC.U4CSE24561
3	Sri Ram	CB.SC.U4CSE24564
4	Tarun Manikanta	CB.SC.U4CSE24555

STM32F401CCU6 BARE-METAL PROTOTYPE / EMBEDDED SYSTEMS CASE STUDY

Group 8: Automated Gravity-Fed Object Counter and Jam Detection System

STM32F401CCU6 Bare-Metal Prototype

Two optical sensors turn motion into speed, size classes, direction checks and jam alerts.

ADC + DMA | 1 ms TIMING | BLUETOOTH



SENSE → MEASURE → CLASSIFY → ALERT

Make every baggage transit observable

A focused embedded prototype for counting, measuring, classifying and supervising object flow.

01 / WHY

Engineering problem

A line operator needs to know what crossed the sensing point, how fast it moved, whether it is the correct direction and whether the next item is late.

02 / WHAT

Embedded objectives

Acquire two analog sensors; calculate velocity; estimate physical length; classify SM / ST / LG; flag reverse motion and speed-adaptive jams; stream telemetry.

03 / SCOPE

Prototype boundary

One tracked object at a time. No motor-speed control, diverter actuator, persistence or production safety interlock. The chute validates the firmware event logic.

SUCCESS CRITERIA

FORWARD
S1 → S2 increments total and reports speed

SIZING
Trailing edge produces length and category

REVERSE
S2 → S1 increments violations, not total

JAM
Missing S2 raises alert and drives buzzer

RESET
PB5 clears state and confirms over UART

Design principle: keep time-critical detection in SysTick; keep blocking UART formatting in main().

Every part has one clear job in the prototype

Controller, sensors, communication, alarm, programming, power and mechanical test hardware.

ELECTRONICS + SIGNALS

QTY	COMPONENT	ROLE / CONNECTION
1	STM32F401CCU6 Black Pill	Main controller · ADC1 + DMA2 + SysTick + USART2 + EXTI5
2	TCRT5000 IR reflective modules	AO → PA1 (S1 entry), AO → PA4 (S2 exit) · VCC = 3V3 · DO unused
1	HC-05 Bluetooth regulated carrier	PA2 TX → RXD · PA3 RX ← TXD · 5 V VCC · 3.3 V UART · 9600 baud
1	5 V active buzzer	Alarm load on the 5 V rail · switched through the NPN driver
1	BC547 / 2N2222 NPN transistor	PB0 → 1 kΩ → base · low-side switch · verify B/C/E pinout
1	1 kΩ resistor	Limits the transistor base current between PB0 and the NPN base

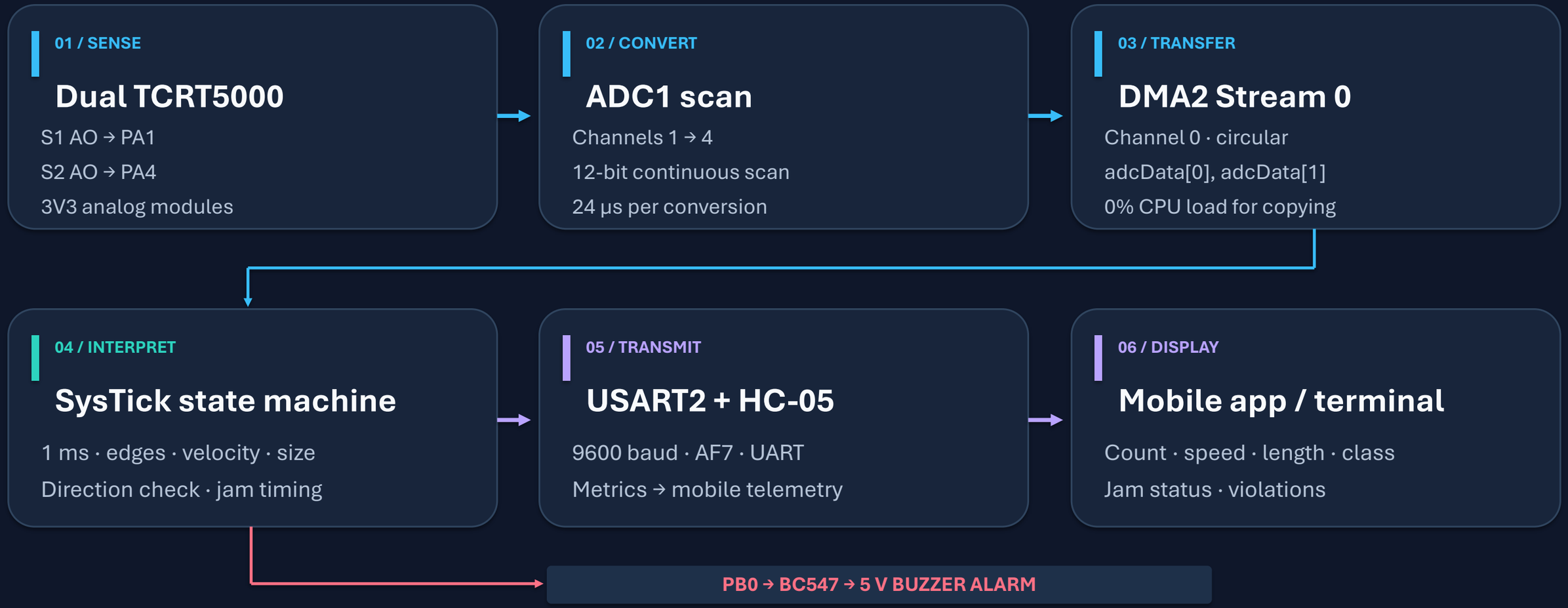
RIG + POWER + DEBUG

QTY	COMPONENT	ROLE / CONNECTION
1	Momentary push button	PB5 → button → GND · internal pull-up · EXTI5 reset input
1	ST-LINK/V2	Programming/debugging · SWDIO → PA13 · SWCLK → PA14 · GND
1	Solderless breadboard	Mounts Black Pill, HC-05, buzzer driver and shared power rails
1 set	Jumper wires	Power, UART, sensor AO, button, buzzer base and common-ground links
1	USB-C cable + USB source	Feeds the Black Pill and 5 V rail; board regulator provides 3V3
1	45° cardboard chute + side guides	Gravity motion source · sensors mounted 150 mm apart along the slope
3+	Test balls / props	Small <50 mm · Standard 50–120 mm · Large >120 mm

POWER RULE: both TCRT5000 modules use 3V3 only. All modules share GND; PB0 never drives the buzzer directly.

From analog reflection to an operator decision

Continuous acquisition feeds a 1 ms state machine; telemetry and alarm outputs branch from it.



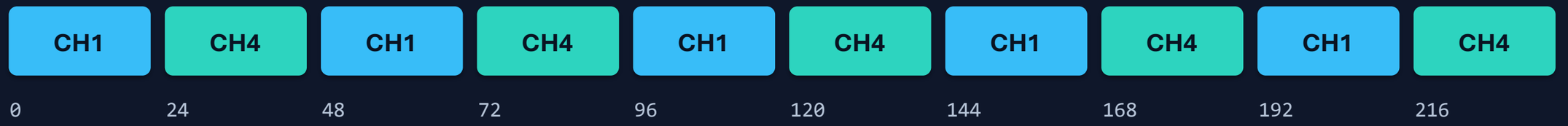
Fast sampling. Deliberate decision timing.

ADC data acquisition and software event detection run at different time scales.

24 µs	48 µs	1 ms	0%
one ADC conversion	one two-channel scan	software edge sampling	CPU load for DMA copying

96 ADC cycles ÷ 4 MHz = 24 µs

Illustrative first 240 µs of a continuous scan



About 21 scan pairs refresh memory per tick. UART formatting remains in main(); detection runs in SysTick.

Seven signal pins. Two supply rails. One common ground.

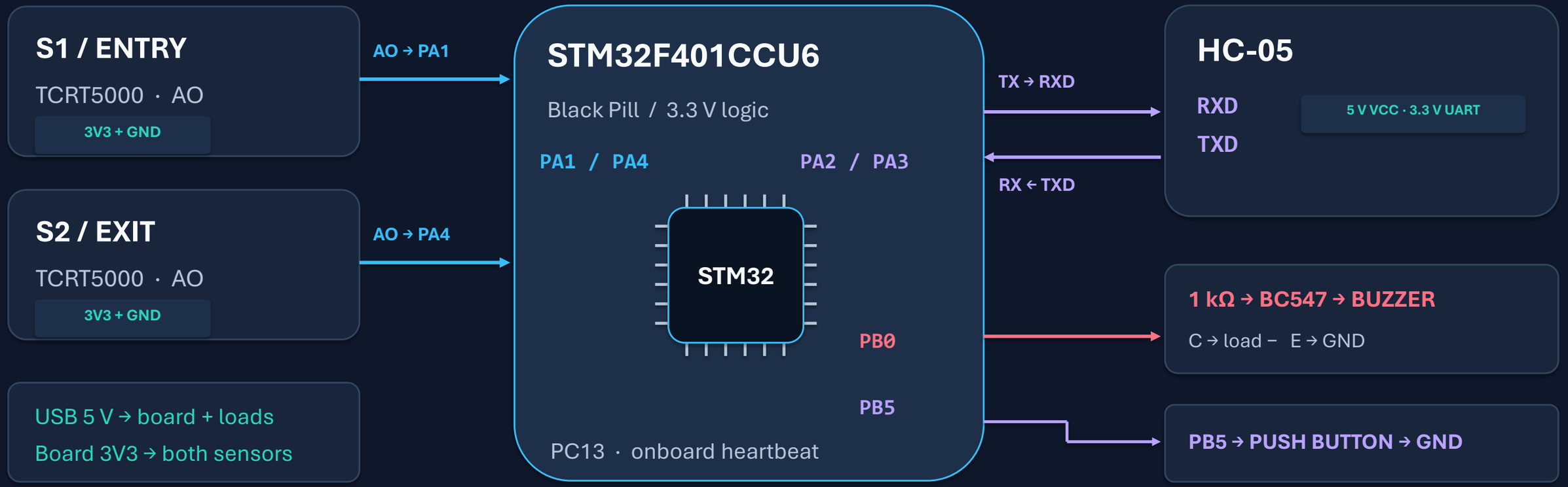
Pin labels are authoritative; physical header order varies across Black Pill and module revisions.

MCU PIN	PERIPHERAL / MODE	CONNECTED HARDWARE	ELECTRICAL DESIGN
PA1	ADC1_IN1 · analog	Sensor 1 AO · entry	0–3.3 V; sensor VCC = 3V3
PA4	ADC1_IN4 · analog	Sensor 2 AO · exit	0–3.3 V; sensor VCC = 3V3
PA2	USART2_TX · AF7	HC-05 RXD	3.3 V UART logic · 9600 baud
PA3	USART2_RX · AF7	HC-05 TXD	3.3 V UART logic
PB0	GPIO output	1 kΩ → BC547 base	3.3 V drive; load = 5 V / ~30 mA
PB5	Input pull-up · EXTI5	Push button → GND	Idle HIGH; falling-edge interrupt
PC13	GPIO output · active low	Onboard heartbeat LED	Onboard current limiting; 1 Hz
3V3 / 5V	Power distribution	Sensors / HC-05 carrier + buzzer	Join all GND connections

> Programming: ST-LINK SWDIO → PA13, SWCLK → PA14, GND → GND; match the target voltage.

Trace each signal from the peripheral to its load

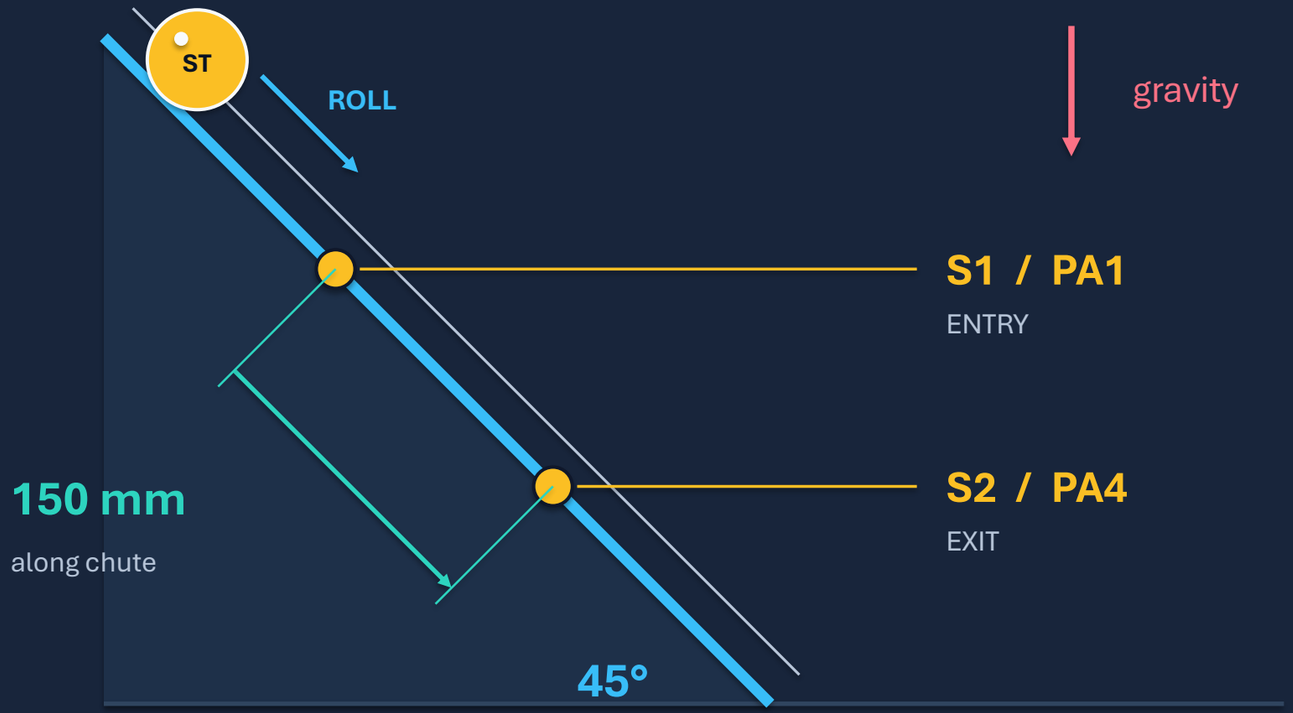
Native connection map • electrical topology, not a physical breadboard layout.



Every module shares GND. Sensors use 3V3; PB0 switches the 5 V buzzer through 1 kΩ + BC547. Leave sensor DO unconnected.

A 45° gravity chute makes the timing visible

Mount two reflective sensors flush with the surface; measure their separation along the direction of roll.



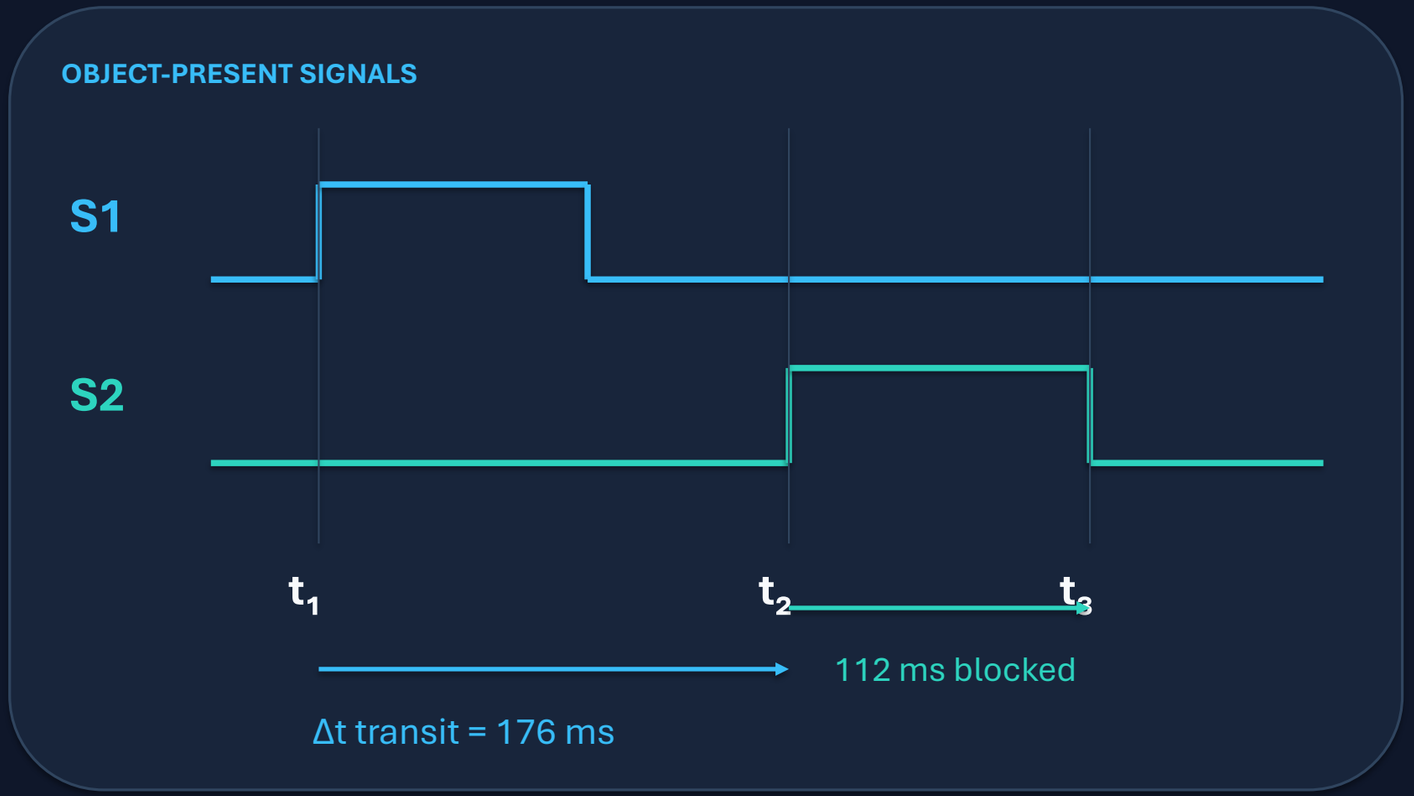
Side guides keep balls over both sensors. Set BELT_GAP_MM to the measured gap.

Leading and trailing edges; three size bands; manual release intervals; reverse motion; partial transit and jam response.

Gravity replaces unavailable conveyor mechanics; the same register architecture and event logic drive the demo.

Use two leading edges, then one trailing edge

Transit time estimates speed. Sensor 2 detection duration converts that speed into an estimated length.



v = 150 × 1000 / Δt

150 × 1000 / 176 = 852 mm/s

length = v × blocked_time / 1000

852 × 112 / 1000 = 95 mm

95 mm → STANDARD (ST)

Worked example. Length is approximate on the accelerating chute; calibrate optical duration against known balls.

One measurement, three handling categories

Small, medium and large balls model the baggage classes at demonstration scale.

SM



Carry-on

< 50 mm

Small test ball

[INFO] LOW SPACING REQ

ST



Standard

50–120 mm

Medium test ball

NORMAL SPACING

LG



Oversized

> 120 mm

Large test ball

[ALERT] HIGH SPACING REQ

50 mm and 120 mm are both ST. These are prototype thresholds; spacing guidance is derived from the category.

GROUP 8: AUTOMATED GRAVITY-FED OBJECT COUNTER AND JAM DETECTION SYSTEM

LIVE WORKING SEQUENCE / One object • two reflective sensors • one complete Bluetooth record

45° GRAVITY CHUTE



150 mm measured baseline

AO → ADC1 → DMA2 → SysTick

1 SENSOR 1 EDGE

Start the transit timer

S1	S2
CLEAR	CLEAR

SPEED	CLASS
—	—

waiting for complete transit...

UART 9600 / 8N1 • HC-05 Classic SPP

How Bluetooth brings the measurements to your phone

The HC-05 bridges the STM32 serial output to an Android phone using Bluetooth Classic SPP.

01 / CONTROLLER

STM32F401



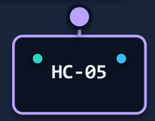
Measures count, speed, length and alarm status.

UART

9600 / 8N1

02 / RADIO BRIDGE

HC-05 module



Receives UART bytes and sends them wirelessly.

Bluetooth

Classic SPP

03 / PHONE

Android app



Receives the text stream and displays the values.

1 Pair the phone

Power the rig; enable Bluetooth.
Select HC-05 in Android settings.
Use the module PIN if prompted.

2 Open the connection

Open a Bluetooth serial terminal.
Choose the paired HC-05 device.
Tap Connect to open its SPP link.

3 Read each update

The terminal shows B,... records.
A dashboard can split each line
and refresh its metrics and alerts.

Data flows from the rig to the phone. Press the physical PB5 button to reset counters; the phone receives RESET.

A live view of the baggage line, on your phone

Proposed Android dashboard • dummy phone with sample data from the project's Bluetooth payload.

01 / BAG COUNT

15 valid bags

Counts forward passages from Sensor 1 to Sensor 2.

02 / TRANSIT SPEED

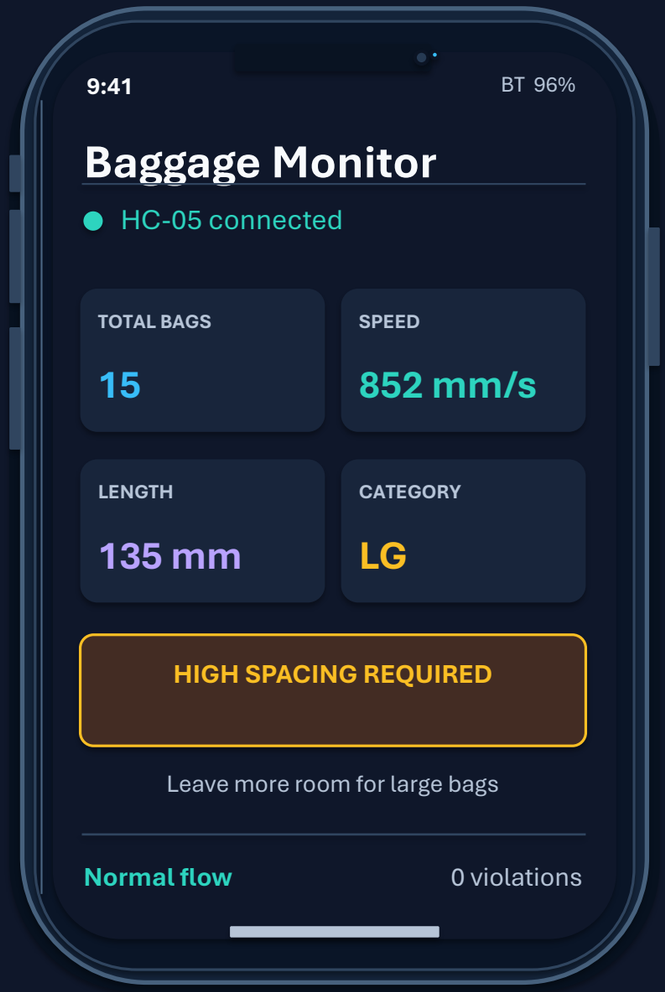
852 mm/s

Uses the 150 mm sensor gap and the leading-edge time.

03 / SIZE CLASS

135 mm → Oversized

Estimated length selects SM, ST or LG handling class.



04 / SPACING GUIDANCE

More space for LG

Class-based advice on the phone: LG = high · ST = normal · SM = low.

05 / JAM OR INACTIVITY

Show an alarm banner

When belt_status becomes 1, highlight the line for inspection.

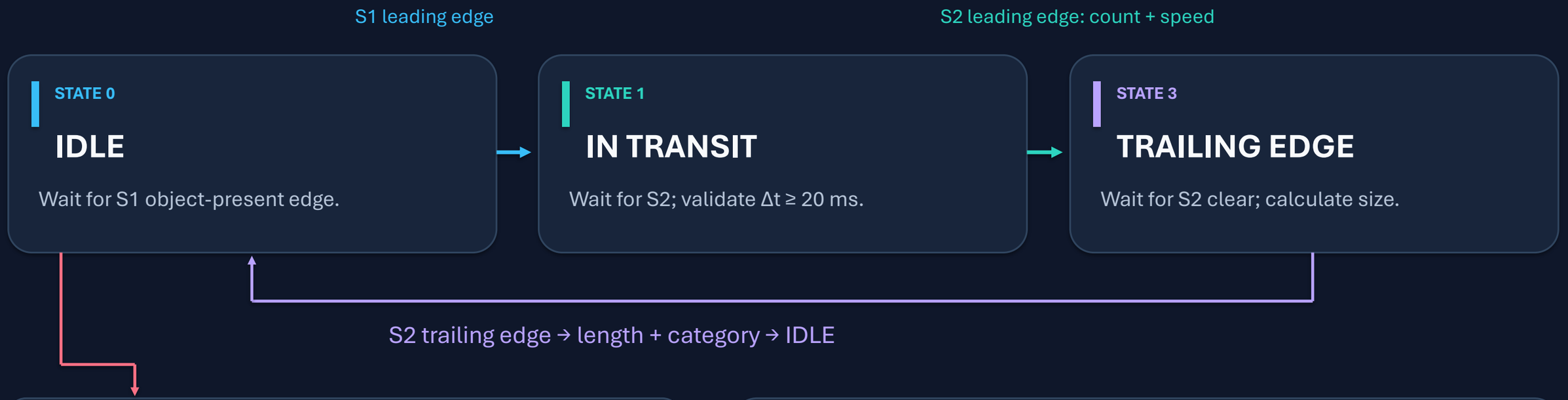
06 / WRONG-WAY MOTION

Flag a new violation

When the violation count rises, notify the operator on the phone.

Four states keep the measurement sequence explicit

SysTick reads hysteretic sensor states every 1 ms; the main loop formats and transmits telemetry.



S2 first → violation; absorb the following S1 edge.

Hysteresis + recovery

Present: ADC < 1800 · Clear: ADC > 2200

PAIR_MS = 2000 bounds pending states; transit timeout raises jam.

Let the last valid speed set the next transit deadline

The controller compares the elapsed S1-to-S2 wait against twice the predicted transit time.

LAST VALID SPEED

1000 mm/s

Illustrative completed pass

EXPECTED TRANSIT

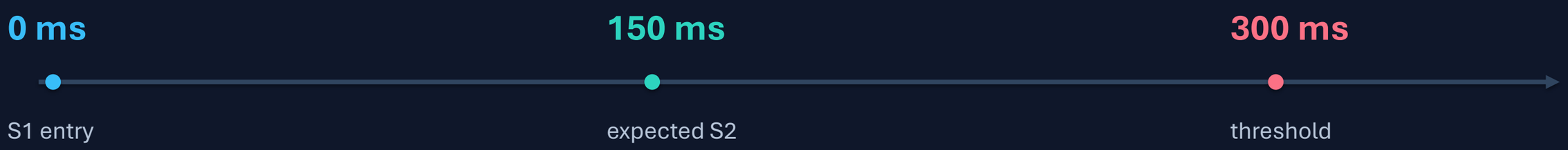
150 ms

150 × 1000 / 1000

JAM THRESHOLD

> 300 ms

2 × expected transit



2 s pairing cap

Missing S2 also raises jam at the pairing timeout.

10 s inactivity alert

No completed transit → idle / stall indication.

PB0 alarm pattern

200 ms ON / 800 ms OFF while alert is set.

Eight checks make the case study reviewable

Bench procedure and expected behavior • complete on the physical rig before claiming measured validation.

CHECK	ACTION ON THE RIG	EXPECTED OBSERVATION
01 / BOOT	Power on; connect Bluetooth terminal	READY message; PC13 heartbeat
02 / IDLE	Allow 10 s without a completed transit	Alert flag; periodic PB0 buzzer pulses
03 / PARTIAL	Trigger S1 only	No count; timeout eventually raises alert
04 / FORWARD	Roll one ball S1 → S2	Count +1; transit velocity reported
05 / SIZE	Roll calibrated small, medium, large props	Estimated length; SM / ST / LG classes
06 / JAM	After a good pass, trigger S1 but no S2	Adaptive timeout raises jam status
07 / REVERSE	Move one ball S2 → S1	Violation +1; valid count unchanged
08 / RESET	Press PB5 button	RESET message; counters clear

> Preparation: measure the 150 mm baseline, calibrate thresholds, confirm 3V3 sensor supply, then test one item at a time.

Why use a gravity chute instead of a conveyor?

The laboratory did not have a motorized chain conveyor; the chute provides a practical motion source.

“We used a 45° gravity chute because a motorized conveyor was unavailable in the lab.
The same STM32 register architecture, timing, sizing and jam-detection logic drive both setups.”

ADC1 scan + DMA2 circular acquisition
1 ms SysTick timing + EXTI button input
Velocity, classification, jam logic + UART

Gravity accelerates the ball between sensors.
Reflective duration depends on geometry and contrast.
One tracked object must clear before the next.

“100% identical” applies to the reused firmware logic; the rig demonstrates principles, with accuracy established by calibration.

Press PB5: clear the tracker and notify the phone

The interrupt requests the reset; the main loop performs the shared-state cleanup safely.

1 PRESS PB5

Button shorts PB5 to GND → LOW

2 EXTI5 FIRES

Handler clears pending bit + debounces 200 ms

3 REQUEST RESET

ISR sets resetFlag = 1

4 MAIN LOOP CLEANS

Counters + state clear; alert turns OFF

AFTER PRESSING THE BUTTON

total	0	violations	0
speed	0 mm/s	bagLength	0 mm
category	-	alert / state	0 / IDLE

MESSAGES TO THE PHONE

```
RESET
```

```
B,0,0,0,-,0,0
```

The interrupt stays short: it requests the reset; the main loop performs the reset and sends the confirmation.

The firmware is deliberately small, bare-metal and inspectable

Peripheral bring-up happens once; the interrupt owns timing while the main loop owns formatting.

BOOT ORDER

01 Clocks + GPIO
- PA1/PA4 analog
- PA2/PA3 AF7
- PB0/PB5/PC13

→

02 DMA2 ready
- Stream 0
- ADC1 request 0
- Circular buffer

→

03 ADC1 scan
- CH1 → CH4
- 84-cycle sample
- Continuous

→

04 USART2
- BRR 0x0683
- 9600 baud
- HC-05 bridge

→

05 SysTick
- LOAD 15999
- 1 ms interrupt
- Start detection

OWNERSHIP SPLIT

SysTick_Handler()
- read hysteretic ADC states
- timestamp edges + compute speed
- classify trailing edge
- apply jam / buzzer logic

main()
- format B,... telemetry
- transmit over blocking UART
- apply resetFlag cleanup
- send READY / RESET

CONFIGURATION EVIDENCE

BLOCK	IMPLEMENTED SETTING
`ADC1 + DMA2`	2 conversions · NDTR=2 · circular · 16-bit
`SysTick`	16 MHz / 16,000 = 1 kHz · CTRL=7
`USART2`	AF7 · BRR=0x0683 · 9600 baud
`EXTI5`	PB5 pull-up · falling edge · 200 ms debounce

> Register-level design keeps the data path visible: ADC1 → DMA2 → SysTick → state machine → UART / alarm.

Complete the engineering story with evidence and limits

What is implemented, what is documented as tested, and what must still be measured on the physical rig.

01 / DELIVERED

Implemented in firmware

ADC1 + DMA2 + SysTick

Speed, size and direction logic

Jam, buzzer and Bluetooth output

02 / CHECKED

Logic verification record

Logic checks cover forward, boundaries, reverse, timeouts, reset and jam.

Record: 13 groups, 0 failures.

03 / VERIFY ON RIG

Bench evidence still required

Calibrate the gap and thresholds. Capture forward, reverse, jam and reset trials on the rig before presenting physical results as measured performance.

KNOWN LIMITATIONS

- One pending object; no queue for close-following bags
- Average gap speed; acceleration changes length accuracy
- RAM-only counters; prototype thresholds; no actuator

NEXT ENGINEERING STEPS

1. Add a pending-object queue and encoder / motor control
2. Add a dashboard, persistent logging and interlocks
3. Validate against known dimensions and a real conveyor

Status: complete embedded prototype and demonstration plan; calibration and physical acceptance testing are the next evidence step.

A traceable project, from register write to system behavior

The presentation is grounded in the local firmware, wiring documentation and STM32 course references.

LOCAL SOURCE	USED FOR / EVIDENCE
`src/main.c`	Register-level implementation, state machine, telemetry format and alarm logic
`README.md`	Project purpose, BOM, pinout, protocol and evaluator defense
`docs/IMPLEMENTATION_GUIDE.md`	Bring-up, wiring, calibration, verification record and limitations
`docs/wiring-diagram.svg + power-wiring.svg`	Signal connections, power rails and safety constraints
`RM_0383_reference_Manual.pdf`	STM32F401 register-field cross-checks
`course_ppt_for_ref/interrupts + lab4 + UART`	Course patterns for SysTick, EXTI, ADC and USART

TAKE-AWAY **Two analog edges become a speed, a size class, a direction decision and an operator alert.**

Questions / discussion